16/09/25

Exp-01 Basic Networking Commands

Aim:

Study of various network commands used in Linux and Windows.

arp-a :

Displays the Address Resolution Protocol (ARP) table which contains the IP address and their corresponding MAC (Physical) address restored in the System.

Output:

Interface : 192.168.56.1 --- 0xd

Internet Address	Physical address	Type
192.168.56.255	ff-ff-ff-ff-ff-ff	Static
224.0.0.22	01-00-5e-00-00-16	Static
224.0.0.251	01-00-5e-00-00-fb	Static

Hostname:

Displays the hostname (computer name) of the local machine.

Output:

LAPTOP-J8UHOT4B

Ipconfig /all :-

Displays detailed information about the network adapter including IP address, Subnet n default gateway, DNS server, DHCP server, MAC address etc.

Output:

Windows IP Configuration

Hostname LAPTOP-J8UHOT4B
Primary DNS Suffix .....
Node Type ..... Mixed
IP Routing enabled ..... NO

nbtstat -a:

Displays the NetBIOS name table of a remote or local computer and help trouble shoot NetBios name resolution problems.

Output:

Ethernet 5:
Node IP address: [192.168.56.1] Scope ID:[]
Host not found

Ethernet:
Node IP address: [172.16.11.50] Scope ID: []
Host not found

netstat -r:

Displays active TCP / UDP connections, listening Ports, routing Table, and network Statics.

Output:

Interface List
18...14 16 9e 6d 9... ab... Realtek PCIe GbE family Controller

13...0a 00 27 00 00 0d..... VirtualBox Host-only Ethernet Adapter

10...54 14 f3 7d c8 d8..... Microsoft Wi-Fi Direct Virtual Adapter #3

23...54 14 f3 7d c8 d8..... Intel(R) Wireless-AC 9560 160MHz

nslookup google.com:

Queries the DNS server to obtain the IP address of domain name.

Output:

Server: Unknown
Address: 172.16.8.1
Non-authoritative answer:
Name: google.com
Addresses: 2404:6800:4007:805::200e
142.250.67.46

Pathping google.com:

Combines the features of Ping and Tracert to trace the path to a destination and calculate Packet loss at each hop.

Output:

Tracing route to google.com [142.250.67.46] over a maximum of 30 hops:

0 LAPTOP J8U HOT4B [172.16.11.50]
1 172.16.8.1
2 187.97.168.129
3 * * *

Ping google.com

Check whether another computer or website is reachable over the network using ICMP Echo Request and ECHO Reply messages.

Output:

Pinging google.com [142.250.67.46] with 32 bytes of data:

Reply from 142.250.67.45: bytes = 32 times = 4MS
TLL = 118

Reply from 142.250.467.46 bytes = 32 time = 10ms
TLL = 118

Reply from 142.250.67.46 bytes = 32 time = 4MS
TLL = 118

route:

Displays or modifies the Ip routing table used by the operating System.

output:

Manipulates network routing tables

Route [-f] [-P] [-4|-6] command [destination] [MASK netmask] [gateway] [METRIC metric]

clears the routing table of all gateway entries. If this is used in conjuction with one of the commands, the table has cleared

mtr google.com:

Combines ping and Traceroute to display the network path and performance statistics.

output:

LAPTOP-J8UHOT4B (172.17.197.94) → google.com

	HOST.	Loss %	Snt	Last
1	LAPTOP-J8UHOT4B (mshome.net)	0.0%	128	0.7
2	192.168.11	0.0%	146	4.7
3	100.84.0.1	0.0%	168	5.7
4	(waiting for reply)			

tcpdump -i eth0:

Captures and displays network packets passing through a network interface for analysis

Output:

tcpdump: verbose output suppressed, use -v or -vv for full protocol decode

listening on the eth0, link type EN10MB (Ethernet) Capture size 262144 bytes.

10:15:21:128456 IP 192.168.1.10.52134 > 8.8.8.53: UDP, length 82

Ping google.com:

Sends ICMP Echo request packets to verify network connectivity and measure response time.

Output:

PING google.com (142.251.43.206) 56(84) bytes 64 bytes from lcmaaa-bb-in-f14.1e100.net (142.251.43.206): icmp-seq=1 ttl=117 time=5.76 ms

Result:

Thus the various windows and Linux networking commands were executed successfully.

20/7/26

Exp-02- Study of different types of network cables

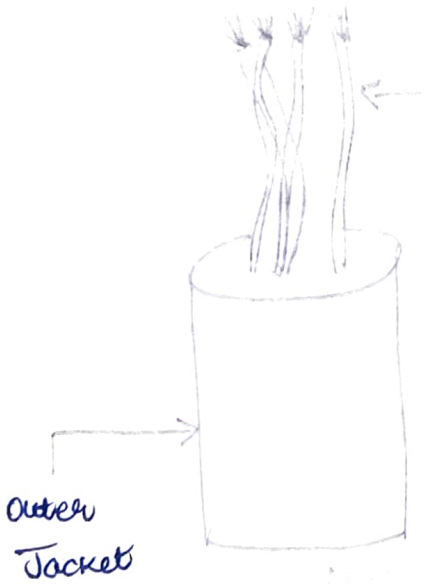
a) Understand different types of network cables:

Different type of cable used in networking as

1. Unshield Twisted pair (UTP) cable
2. Shield Twisted pair (STP) cable
3. Coaxial Cable
4. Fibre optic cable

1. UTP cable:

Category 3 - 10 mbps
Category 5 - upto 100 mbps
Category 5e - 1 Gbps



Advantages:

* Cheaper in cost
* Easy to install as they have a smaller overall diameter.

Disadvantages:

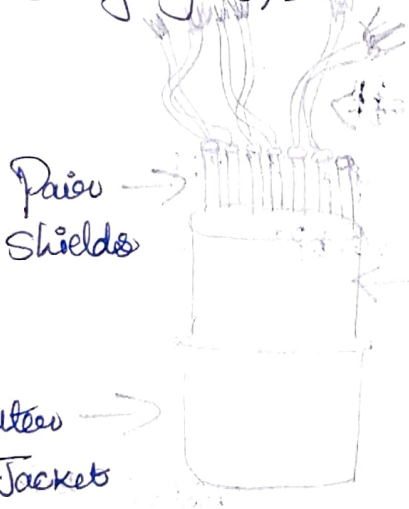
* More prone to (EMI) Electromagnetic interference and noise.

Applications:

* 10 Base - T ethernet
* Fast Ethernet, Gigabite Ethernet

2. STP:

Category 6, 6a - 10 GbPS



SSTP

Category 7-10 GbPS

Advantages:

* Shielded
* Faster than UTP
* Less Susceptible to noise and interfaces

Disadvantages:

* Expensive
* Greater installation effort

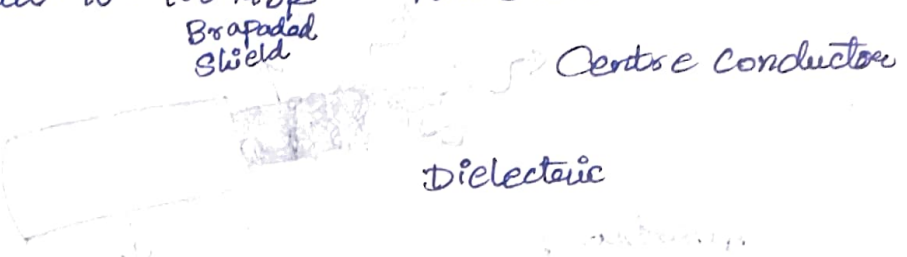
Applications:

* Gigabit Ethernet, 10G Ethernet (55m) widely used in data centers.
* Gigabit Ethernet, 10G Ethernet (100m)

3. Coaxial cable:

RG-6
RG-59
RG-11

Speed - 10 - 100 Mbps



Advantages:

* High bandwidth
* Immune to interferance
* Low Less bandwidth
* Versatile

Disadvantages:

* Limited distance
* Cost
* Size is bulky

Applications:

* Speed of Signal is 500m
* Television network
* High Speed internet Connections

4. Fibre optic cable :

Signal mode
Multi mode — 100 Gbps



Advantages :

* High Speed
* High bandwidth
* High security
* Long distance

Disadvantages :

* Expensive
* Requires skilled installers

Application :

* maximum distance of fibre optics cable is around 100 meters.

Result :

Hence studied all the types of cables.

Exp-03

Experiments on Cisco Packet Tracker

Aim:

To Study the Packet tracker tool Installation and user interface overview.

Introduction:

A Simulator, as the name Suggests, Simulates network devices and its environment. Packet tracker is an exciting network design, Simulation and modelling tool.

(i) It allows you to model complex systems without the need for dedicated equipment.

(ii) It helps you to practice your network Configuration and trouble Shooting Skills via Computer or on Android or ios based mobile devices.

(iii) It is available for both the Linux and windows desktop environments.

(iv) Protocols in packet tracer are coded to work and behave in the Same way as they would on real hardware.

Analyse the behaviour of network device using Cisco packet Tracer Simulation:

1. From the network Component box click and drag-and-drop the below components.

a) 4 Generic PCs and one HUB

b) 4 Generic PCs and one Switch

2. Click on Connection:

a. click on Cooper - Straight Through Cable
b. Select one of the pc and Connect it to HUB using the Cable. The link is up, Similarly Connect remaining 3PCs to the HUB.

C. Similarly, connect 4PCs to the Switch using copper Straight - through Cable.

3. Click on the PCs Connected to hub, go to the Desktop tab, click on Ip Configuration, and enter an Ip address and Subnet mask. Here, the default gateway and DNS Server information is not needed as there are only two end devices in the network.

Click on the PDU (message icon) from the Common tool bar,

a) Drag and drop it on one of Pc and then drop it on another PC connected to the HUB.

4. Observed the flow of PDU from Source PC to destination PC by selecting the real time mode of Simulation.

5. Observe how the HUB and Switch are forwarding the PDU and write your observation and Conclusion about the behaviours of Switches.

Result:

CISCO Packet Tracer is learned and executed Successfully.

27/7/26

Exp - 04

Setting up a LAN

Aim:

To design, setup, and configure a local area network (LAN) connecting four computers through a network switch, assign static IP addresses to all devices, verify connectivity using the ping command and demonstrate resource/data sharing across the network.

Implementation in Cisco Packet Tracer:

Since physical hardware was limited, the LAN was also implemented and verified using CISCO packet Tracer Simulation Software.

Step-1 : place the devices

1. open cisco packet tracer
2. From the device category panel, select End devices and drag 4 PCs onto the workspace.
3. Select switches and drag 1 switch onto the workspace.

Step 2 : Cable the devices

4. Click Connections → Select Copper, Straight-through Cable

5. Click each PC, then Click the Switch and Select an available Fast Ethernet Port to complete each connection.

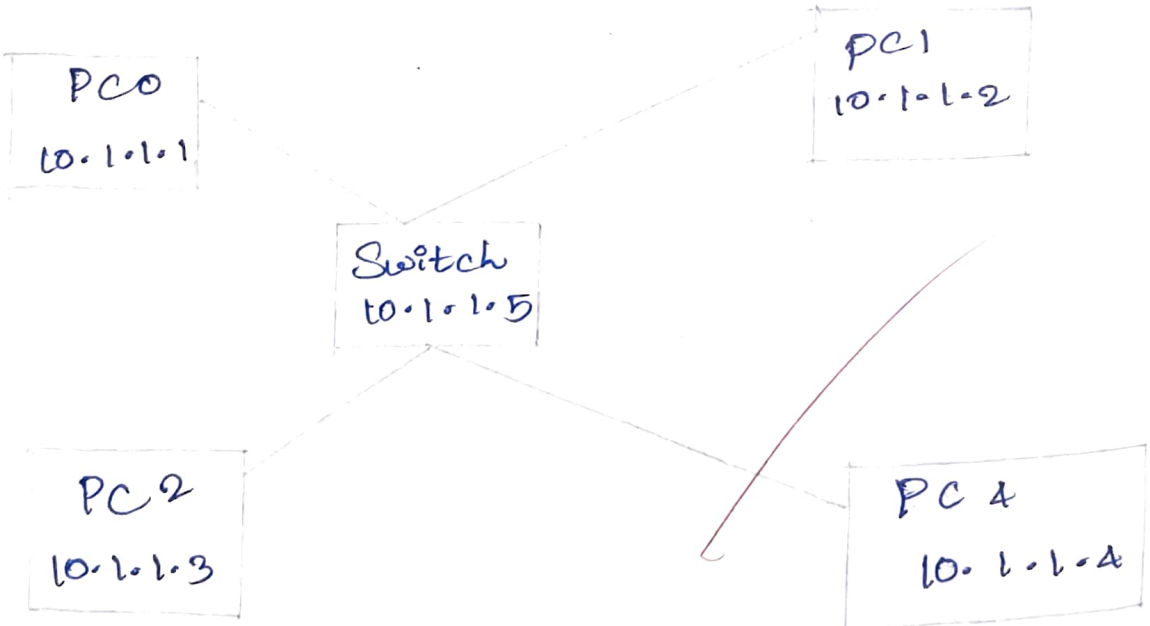
6. Repeat for all the PCs green link, confirms a Successfull physical Connection.

Equipment Required:

S.no	Equipment	Quantity	Specification
1	Personal Computer (PC)	4	Ethernet (NIC) enabled
2	Network Switch	1	8/16/24 Port, unmanaged / managed
3	Ethernet cables	4	Straight - through, RJ-45 Connectors

Network diagram:

Start topology LAN



IP Address Configuration Table :

Device	IP address	Subnet mask	Connected to
PC_0	10.1.1.1	255.0.0.0	Switch port Fa0/1
PC_1	10.1.1.2	255.0.0.0	Switch port Fa0/2
PC_2	10.1.1.3	255.0.0.0	Switch port Fa0/3
PC_3	10.1.1.4	255.0.0.0	Switch port Fa0/4
Switch (management)	10.1.1.5	255.0.0.0	VLAN 1 Interface

Step 3: Assign IP Addresses to PC

7. Click a PC → Desktop tab → IP Configuration
8. Enter the Static IP addresses and Subnet Mask as per the IP table for each PC.

Step 4: Configure the Switch management IP

9. click the Switch → CLI tab
10. Enter the following Commands to assign a management IP:

```
enable
configure terminal
interface VLAN1
ip address 10.1.1.5 255.0.0.0
no shutdown
exit
```

Step 5: Verify Connectivity using Ping

11. click a PC → Desktop tab → Command Prompt.
12. Use the Ping Command to test Connectivity to the other PCs eg: Ping 10.1.1.2
13. Successful "Reply from...." messages confirm Connectivity.

Step 6: Demonstate Data Sharing

14. Add a server device and Connect it to the switch.

15. Assign it an IP address via its Desktop → IP Configuration table.

16. Open the Servers Services tab and enable the HTTP Service.

17. From another PC, Open the Web browser and enter the Servers IP address and to load the page, Confirming Successfull data transfer accross the LAN.

Step 7: Observe Packet flow in Simulation mode

18. Switch to Simulation mode.

19. Send a Ping or web request between two devices and use the Play/Capture controls to step through the Switch, observing MAC address based forwarding.

Result:

Thus setting up LAN was executed Successfully.

03/8/26

Exp-05 : Experiment on Packet capture tool - Wireshark

Aim: Experiments on Packet Capture tool : Wireshark

Wireshark :

Wireshark a network analysis tool formally known as Ethereal, Captures packets in realtime and display them in human-readable format. Wireshark includes filters, colour coding and other features that let you dig deep into network traffic and inspect individual packets.

Procedure :

Capfusing Packets :

After downloading and installing Wireshark launch it and double click the name of the network interface where Capture to start capturing Packets on that interfaces. As soon as you click the interfaces name you will see the packets start to appear in the real time. Wireshark Capture each packet sent to or from your system. If you have promiscous mode enabled it enable by default you will also see all the other side Packets on the network instead of only Packets addressed to your network. To check of options and verify the "Enable Promiscous mode on all Interfaces". check is activated on window and at the bottom. Create a filter to display only TCP Packets, inspect the packets and Provide the flow graph.

Procedure:

* Select local area Connection in Wireshark
* Go to Capture option
* Select Stop Capture automatically after no Packets.
* Then click Start Capture
* Search TCP packets in search box
* To see flow graph click Statistics flow graph.
* Save the packets

Create a filter to display only ARP packets and inspect the packets.

Procedure:

* Go to Capture option
* Select Stop Capture automatically after no packets.
* Then click Start Capture.
* Search ARP Packets in Search box
* Save the Packets

Results :

Thus the Packet capture was executed Successfully using Wireshark.

Exp-06 : Hamming Code

Aim:

To write a Program to implement error detection and correction using hamming Code concept.

Sender.py:

```
def find_r(k):
    r=0
    while 2**r < k+r+1
        r+=1
    return r

text = input ("Enter text:")
binary = " "
for ch in text:
    binary += format (ord(ch), "08b")
Print ("Binary:", binary)
k=len (binary)
r= find_r(k)
Print ("Data bits:", k)
Print ("Redundancy bits:", r)
n= k+r
Code = [0]*(n+1)
j =0
for i in range (1, n+1):
    If i & (i-1) != 0:
        Code[i] = int (binary[j])
        j+=1
p=1
```

```
While p <=n:
  Parity=0
  for i in range (p,n+1):
      if i & p:
        Parity ^= code [i]
    code [p] = parity
    p = p* 2
hamming- code = " "
for i in range (1,n+1):
      hamming_code += str (code [i])
Print ("Hamming code :", hamming_code)
with open ("channel. txt","w") as f:
      f. write (str(k) + "\n")
      f. write (hamming_code)
Print ("Data Saved in channel. txt")
```

Receiver.py

```
def binary to text (binary):
      text = " "
      for i in range (0, len (binary), 8):
        byte = binary [i: i+8]
        if len (byte) == 8:
        text+ = chr (int (byte, 2))
      return text
with open ("channel txt ","r") as f:
      k= int (f readline ()).
      data= f. readline (). strip ()
```

```
Print ("Received Hamming Code :", data)
n = len(data)
Code = [0] + [int(x) for x in data]
Corrupted_binary = ""
for i in range(1, n+1):
    if i & (i-1) != 0:
        Corrupted_binary += str(code[i])
Corrupted_binary = Corrupted_binary[:k]
Corrupted_text = binary_to_text(Corrupted_
                                binary)
Print ("Corrupted Text:", Corrupted.text)
error = 0
P = 1
while P <= n:
    Parity = 0
    for i in range(P, n+1):
        if i & P:
            Parity ^= code[i]:
        if Parity:
            error += P
        P = P * 2
if error == 0:
    Print ("No error detected:")
else:
    Print ("Error detected at Position:" error)
    Code[error] = 1 - Code[error]
    Print ("error occured.")
```

```
Corrected_binary = " "
for i in range (1, n+1):
    if i & (i-1) != 0:
        Corrected_binary += str (code[i])
Corrected_binary = Corrected_binary [:k]
Corrected_text = binary_to_text (corrected_binary)
Print ("Corrected Text : ", Corrected_text)
```



Result:

Thus error detection and correction is done using hamming code.

Exp-7

10/8/96

Flow Control ADD Data Link Layer

Aim:

Write a program to implement flow control at data link layer using SLIDING WINDOW protocol. Simulate the flow of frames one node to another.

Sender.py

```
import time
import os
def save_data (file_name, data):
    with open (file_name, "w") as file:
        file.write (data)

def get_data (file_name):
    if not os.path.exists (file_name):
        return None
    if os.path.getsize (file_name) == 0:
        return None
    with open (file_name, "r") as file:
        return file.read().strip()

def empty_file (file_name):
    if os.path.exists (file_name):
        open (file_name, "w").close()

def Sender_Program():
    Print ("=======")
    Print ("SENDER PROGRAM")
    Print ("=======")
size = int (input ("Enter window size:"))
text = input ("Enter window size:")
length = len (text)
```

```
Start = 0
empty_file ("Sender_Buffer.txt")
empty_file ("Receiver_Buffer.txt")
While Start < length:
    end = min (Start + size, length)
    frames = ""
    Print ("\n ---------")
    Print ("Sending Frames")
    Print ("-------")
    For position in range (Start, end):
        frame_number = position
        Character = text [position]
        Print (
            "Frame No:", frame_number,
            "Data:", Character)
        frames += f "{frame_number}:
                    {character}\n"

    Save_data (
        "Sender_Buffer.txt", frames.strip())
    Print ("\n Sender_Buffer.txt updated")
    Print ("\n waiting 10 seconds...")
    Print ("you can manually change a
        frame number", "in Sender_Buffer.txt")

    time.sleep (10)
    reply = None
    for attempt in range (5):
        reply = get_data ("Receiver_Buffer.txt")
        if reply:
            break
```

```
    time.sleep(1)
    if reply:
        Print("\n Receiver Response ", reply)
```

Receiver.py

```
import time
import os
def save_response(file_name, data):
    with open(file_name, "w") as file
    file.write(data)
def get_response(file_name):
    if not os.path.exists(file_name):
        return None
    if os.path.getsize(file_name)==0
        return None
    with open(filename, "r") as file
    return the read() strip()
def clean_buffer(file_name):
    if os.path.exist(file_name)
    open(filename, ".w").close()

def receiver_program()
    Print("=======")
    Print("RECEIVER PROGRAM")
    Print("=======")
    next_frame=0
    message_data=[]
    Print("\n receiver is waiting for frame
```

```
While true
    time.sleep(1)
    incoming_data = get_response("Sender_Buffer.txt")
    if not incoming_data:
        Continue
    if incoming_data == "Done":
        Print("\n Transmission Completed by
              Sender.")
        Print("Final Message :", "join(message_data)")
        break

if __name__ == "__main__"
    receiver - program
```

Output:

```
==========
 SENDER PROGRAM
==========

Enter Window Size : 3
Enter Text Message : hello

----------
 SENDING FRAMES
----------

Frame NO : 0        DATA : h
Frame NO : 1        DATA : e
Frame NO : 2        Data : l

Sender_Buffer.txt updated
```

Waiting 10 sec....
You can manually change a frame number in Sender_Buffer.txt

==========

RECEIVER PROGRAM

==========

Receiver is waiting for frames

Receiver ~~is waiting~~ Sender buffer.txt

Frame 0 Data : h => correct

Frame 1 Data : e -> correct

Frame 2 Data : l -> Correct

Sending ACK : 2

Received message : hel

Results :

Thus the Sliding Window Protocol was executed Successfully implemented using Python.

3/8/26

Exp-8 Nmap using Try HackMe

Aim:

To discover live host on a network using Nmap host discovery scans - ARP scan, ICMP scan and TCP/UDP ping scan and to understand which protocol operates on which layer of TCP/IP model and when each one applies.

ARP host Discovery

```
$ Sudo nmap -PR -Sn 10.200.6.0/24
```

Output

```
Starting Nmap 7.80 (https://nmap.org)
Nmap Scan report for 10.200.6.1
Host is up
Nmap done : 256 IP address (1 host up) scanned
  in 105/s
```

ICMP host Discovery

ICMP Echo (-PE)

```
$Sudo nmap -PE -Sn 10.200.6.0/24
nmap Scan report for 10.200.6.1
Host is up (0.069s latency)
Nmap Scan report for 10.200.6.50
Host is up (0.070 latency)
Nmap Scan report for 10.200.6.250
Host is up (0.069s latency)
Nmap done : 256 IP address (3 has to up)
Scanned in 3.38 Seconds
```

ICMP Timestamp (-PP)

```
$ nmap -PP -Sn 10.200.6.0/24
Nmap Scan report for 10.200.6.50
Host is up (0.069s latency)
```

Nmap Scan report down 10.200.6.25:
Host is up 3.0s latency
Nmap done: 256 IP address (3 hosts up) in
3.64 seconds

IP address mask (-PM)

$ nmap -PM -sn 10.200.6.0/24
Nmap done: 256 IP address (3 hosts up) Scanned
in 55.14 seconds

TCP UDP ping Discovery

(i) TCP SYN ping (-PS)

$ nmap -PS -sn 10.200.6.0/24
Nmap Scan report for 10.200.6.50
Host is up (0.073 latency)
Nmap Scan report for 10.200.6.250

(ii) TCP ACK ping (-PA)

Sends a TCP packet with the ACK flag set
$ sudo nmap -PA -sn 10.200.6.0/24
Nmap Scan report for 10.200.6.50
Host is up (0.070s latency)
Nmap Scan report for 10.200.6.250
Host is up (0.0693 latency)

(iii) UDP ping

$ sudo nmap -PU -sn 10.200.6.0/24
Nmap Scan report for 10.200.6.50
Host is up (0.070 Latency)
Nmap Scan report for 10.200.6.250

Host is up (0.069 latency)
Nmap done: 256 IP addresses (9 hosts up) Scanned in 35.97s

Masscan - A Router Alternative:

$ masscan 10.200.6.0/24 -p443
$ masscan 10.200.6.0/24 -p80,443
$ masscan 10.200.6.0/24 -p22-25

Results:

Thus like host discovery for ARP, ICMP, TCP UDP was studied and the output in verified Successfully.

17/8/26

Exp-09 Implementation of Subnetting in CISCO Packet Tracer Simulation

Aim:

To implement the Subnetting in CISCO Packet Tracer Simulator.

Procedure:

* open CISCO packet Tracer and create a new network technology.
* Add the required router, switches and PCs to workspace.
* Connect the device using appropriate copper cables.
* use the network 192.168.1.0/24 and divide into 12 T network.
* Configure the router interface with the required IP address and Subnet mask 255.255.255.224
* use enable and Configure terminal in the router CLI
* Enter the Interface Configuration terminal and assign IP using
  (i) ip address <IP address> 255.255.255
  (ii) no Shutdown
* Configure the required Switch port in access mode
* use the Ping Command to test connectivity between PCs and the routers.

* The ping is Successfull, the network is Configured Successfully.

Result:

The ping is the network is configured Successfully.

Exp-10 Inter Networking with routers in CISCO Packet Tracer Simulator

Aim:

To design and configure a simple Internetwork using a router in CISCO packet tracer and test connectivity between two PCs

Procedure:

Step 1:

Configure routers

* Select the routers and open CLI
* Press enter to start Configure routers.
* Type enable to activate the Privilaged mode.

Step 2:

* Assign IP address to every PC in the network.
* Select the PC, Go to desktop and Select IP Configuration and assign IP address default Gateway, Subnet mask
* Assign the default gateway of PC is 192.168.10.1
* Assign the default gateway of PC as 192.168.20.1

Step 3 :

* connect fast Ethernet 0 port of with fast ethernet 0/0 port of rout using a Copper Straight - through

* Connect Fast Ethernet 0 port of with fast Ethernet 0/0 port of router a Copper Straight - through Cable.

Results :

Thus we designed and configured a S interconnection using a router i.e. CISCO packet

21/8/96

Exp-11 Design and Configure an internetwork using wireless router, DHCP server and internet

Aim:

To design and Configure an internetwork using a wireless router, DHCP server and internet cloud in CISCO Packet Tracer.

Procedure:

- Open CISCO Packet Tracer and Create the required network Topology.
- connect pc, laptop, wireless router, cable modem Internet cloud and server.
- Configure the wireless router with SSID home network and Enable DHCP.
- connect the laptop to the windows wireless network and set the pc to DHCP
- configure the server with IP 208.67.220.220 and enable DHCP and DNS
- Verify Connectivity using ip config and ping Cisco-com.

Result:

The Internetwork was Successfully designed and configured using a wireless router, DHCP server and Internet cloud and Connectivity Successfully Verified using Ping Cisco-Com.

10/9/24

Exp-12 End to End Communication at transport layer

Aim:

To implement an echo-client server using TCP/UDP socket and implement a chat client-server using Socket Programming.

a) echo-client-server using TCP/UDP:

a socket is an endpoint used for communicates between two devices. In an echo program, the client sends a message to server and the server ends the same message back to client.

Procedure:

1. Create Server program and client program
2. Import the required socket library
3. In server, creates a socket and bind it is an IP address and port no
4. make the server to the client request
5. Start client and connect it to server
6. enter message from clients
7. send message from clients
8. The server receives the message and sends same message back.
9. The client receives and display the echoed message.
10. close socket connection

b. Client Server using Socket Program

1. Create server and client program using Socket Programming.

2. Start Server and bind it is an IP address and Port.

3. make Server wait for client connection

4. Start client and Connect it to Server

5. Enter message in client

6. Send message to Server

7. The Server recieves and display message.

8. The server sends a reply back to client

9. Continue Sending and recieving message to perform a Chat

10. Close the connection of HV the Chat is completed.

Result:

The end to end Communication was Successfully implemented using TCP/UDP Sockets.

19/9/26

Exp-13 Implement your own Ping Program

Aim:

To implement a simple ping Program to check the Connectivity between two network devices.

Procedure:

1. Create a new program for implementation. ping operation.
2. import the required Socket/network library.
3. Get IP address or hostname of the destination.
4. Create a Socket for Communication.
5. Send req packet to the destination.
6. Wait for response from destination.
7. If a response is received, display that destination is reachable.
8. Reach the process for required number of Packets.
9. Display the result and close the Connection.

Algorithm:

1. START
2. Enter destination Ip address
3. create a socket
4. send a ping request
5. wait for response
6. If response is received
7. otherwise display host is unreachable.
8. STOP

Result:

Thus, the Ping Program was Successfully Implemented.

4/9/26

Exp-14 Raw Sockets – Packet Sniffing

Aim:

To write a program using RAW sockets to implement packet sniffing.

Procedure:

1. open Python IDLE or terminal
2. Create a Python program for packet Sniffing.
3. Import Socket module
4. Create RAW Socket
5. use recvfrom() fun to receive packets
6. Read captured packet data
7. Display packet information
8. Generate some network traffic and observe the captured packet.

Algorithm:

1. START
2. Create a socket
3. Find the Socket to required network interface.
4. Wait for Incoming network Packets
5. Capture the packet using RAW Socket.

6. Extract packet information

7. Display packet detail

8. Stop

Result:

Thus a RAW Socket based on packets sniffing Program was implemented Successfully.

1/10/96

Exp - 15 Analysis of web logs using weblizer tool

Aim:

To analyze the different types of web logs using weblizer tool.

Procedure:

1. Run the weblizer window version
2. Select or provide the required web log file.
3. Download web log file into webalizes
4. Load web log file from web if required.
5. Press Run webalizes
6. Weblizes process the log file
7. observe the generated analysis / report
8. Study the different information available on the report.

input:

A web ser log file

Output:

Web log analyze report generated using webalizes.

Result: 22/X/26

Thus, the web log file was Successfully analyzed using webalizes tool.